AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI



Smart Classroom Energy Management: Occupancy-Driven Lighting and Air-Conditioning Control using IoT and Machine Learning

Report submitted for

INTERNET OF THINGS PROJECT REPORT

submitted by

NIRANJAN N S	AM.SC.U4CSE23338
SIVADEV SMIJITH	AM.SC.U4CSE23359
YALAMARTHI JYOTHI SRI KRISHNA CHAITANYA	AM.SC.U4CSE23370
PENUMALLU RAM DINESH REDDY	AM.SC.U4CSE23341
MANISH SHISHIRCHANDRA DORA ALURI	AM.SC.U4CSE23333

under the guidance of

Dr. Sangeeth Kumar

Associate Professor

Center for Wireless Networks & Applications (Amrita WNA)

Amrita School of Computing

Amrita Vishwa Vidyapeetham

Amritapuri

Academic Year: 2026–2027

Certificate

This is to certify that the project report titled **"Smart Classroom Energy Management: Occupancy-Driven Lighting and Air-Conditioning Control using IoT and Machine Learning"** is a bona fide record of the work carried out by the following students under my guidance, in partial fulfilment of the requirements for the Internet of Things project component of the B.Tech programme, Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, during the academic year 2026–2027.

Name	**Roll Number**
NIRANJAN N S	AM.SC.U4CSE23338
SIVADEV SMIJITH	AM.SC.U4CSE23359
YALAMARTHI JYOTHI SRI KRISHNA CHAITANYA	AM.SC.U4CSE23370
PENUMALLU RAM DINESH REDDY	AM.SC.U4CSE23341
MANISH SHISHIRCHANDRA DORA ALURI	AM.SC.U4CSE23333

Dr. Sangeeth Kumar
Associate Professor
Center for Wireless Networks & Applications (Amrita WNA)
Amritapuri

Dr. Lekha S Nair
Head of the Department
Department of Computer Science and Engineering

Place: Amritapuri
Date: 04/11/2026

Declaration

We hereby declare that the project report titled **"Smart Classroom Energy Management: Occupancy-Driven Lighting and Air-Conditioning Control using IoT and Machine Learning"** is an original record of work carried out by us under the guidance of **Dr. Sangeeth Kumar**, Associate Professor, Center for Wireless Networks & Applications (Amrita WNA), Amrita Vishwa Vidyapeetham, Amritapuri, and that it has not been submitted elsewhere for the award of any degree or diploma. All measured data analysed in this report come from the public ROBOD dataset (Tekler et al., 2022). Every predicted value comes from our models, and every simulated value (actuator states, controlled energy, room simulation) is labelled as simulated. No result has been fabricated, and wherever something could not be verified the report says so.

NIRANJAN N S	AM.SC.U4CSE23338
SIVADEV SMIJITH	AM.SC.U4CSE23359
YALAMARTHI JYOTHI SRI KRISHNA CHAITANYA	AM.SC.U4CSE23370
PENUMALLU RAM DINESH REDDY	AM.SC.U4CSE23341
MANISH SHISHIRCHANDRA DORA ALURI	AM.SC.U4CSE23333

Place: Amritapuri
Date: 04/11/2026

Acknowledgement

We are sincerely grateful to **Dr. Sangeeth Kumar**, Associate Professor, Center for Wireless Networks & Applications (Amrita WNA), Amrita Vishwa Vidyapeetham, Amritapuri, for his steady guidance, constructive feedback and support throughout this project.

Our thanks also go to the faculty and staff of the Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Amritapuri, for their support while the project was under way.

We acknowledge the authors of the ROBOD dataset (Tekler, Lei, Peng, Miller and Chong), whose openly published lecture-room data made this work possible. We also acknowledge the open-source communities behind Eclipse Mosquitto, Node-RED, SQLite, scikit-learn and PyTorch.

Finally, we thank our families and friends for their encouragement.

NIRANJAN N S
SIVADEV SMIJITH
YALAMARTHI JYOTHI SRI KRISHNA CHAITANYA
PENUMALLU RAM DINESH REDDY
MANISH SHISHIRCHANDRA DORA ALURI

Abstract

This project builds an end-to-end Internet of Things system that monitors lecture rooms and controls their lighting and air-conditioning (AC) from occupancy analytics. It follows the eight-stage reference architecture:

- an IoT node replays the real ROBOD dataset (two lecture rooms, 5-minute data, 29 weekdays) to an authenticated Mosquitto MQTT broker;
- Node-RED validates, stores (SQLite), visualises (Node-RED Dashboard 2.0) and decides;
- a Python service runs traditional, ensemble and deep-learning occupancy models;
- simulated light and AC actuators on a second laptop execute the commands and acknowledge them.

Two recent research methods are integrated as adaptations: a Mamba selective state-space occupancy model (the model family of OCC-Mamba) and an LSTM + Dueling Deep Q-Network HVAC controller (Hou et al., 2025).

Model results. On six held-out test dates, Mamba reached the best test F1 (0.651). The random forest, selected by validation F1 before the test dates were scored, reached 0.559. A date-cluster bootstrap shows that the models are not significantly different on this amount of data.

Energy results (all simulated):

- Rule-based control with predicted occupancy used 52% less simulated cooling than always-on operation (405.5 vs 850.6 kWh thermal over six test days), with 0.6% of occupied intervals outside the comfort band.
- On recorded energy re-scaled by our commands, it saved 38.7% of lighting and 39.5% of chilled-water energy, but switched the lights off in 27.9% of occupied intervals. This comfort cost is reported alongside every saving.
- The Dueling DQN saves more energy but violates comfort more often, so rules remain the default.

Engineering results. The live loop acknowledged 100% of commands, with a median round trip of 88 ms. The project has 97 automated tests and an end-to-end test, all passing.

Keywords: Internet of Things, MQTT, Node-RED, occupancy estimation, smart building, energy management, Mamba, Dueling DQN.

Contents

Certificate ii

Declaration iii

Acknowledgement iv

Abstract v

List of Abbreviations x

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Scope . . . 2
1.5 Significance of the Study . . . 2
1.6 Methodology Overview . . . 2
1.7 Organisation of the Report . . . 3

2 Literature Review 4

2.1 ROBOD Dataset . . . 4
2.2 Paper A: OCC-Mamba . . . 4
2.3 Paper B: LSTM and Dueling DQN . . . 4
2.4 Gaps and Need for the Study . . . 5

3 Problem Definition and Requirements 6

3.1 Detailed Problem Description . . . 6
3.2 Project Requirements . . . 6
3.2.1 Functional Requirements . . . 6
3.2.2 Analytical Requirements . . . 6
3.3 Hardware and Software Requirements . . . 7
3.4 Constraints and Assumptions . . . 7

4 System Design and Methods 8

4.1 Dataset Description 8
4.2 System Architecture 9
4.3 MQTT Architecture 10
4.4 Node-RED Flow 11
4.5 Database Design 12
4.6 Dashboard Design 12
4.7 Feature Engineering 15
4.8 Machine-Learning Models 16
4.9 Research-Paper Adaptations 17
4.10 Decision Engine 17
4.11 Dueling DQN 18
4.12 Actuator Simulation 19
4.13 Experimental Methodology 20

5 Results and Discussion **21**
5.1 Model Evaluation 21
5.2 Energy Evaluation 23
5.3 Comfort Analysis 24
5.4 Performance Evaluation 25
5.5 Testing 26
5.6 Two-Laptop Deployment 27

6 Conclusion and Future Work **29**
6.1 Conclusion 29
6.2 Limitations 29
6.3 Future Work 30
6.4 Team Contributions 30

References **31**

A Reproduction Instructions **32**

List of Figures

4.1 Implemented architecture: eight reference stages across two laptops 9
4.2 Node-RED flow (editor, after login) 11
4.3 Dashboard (1/3): control, decision with the deciding layer, sensors, health, live predictions and model performance from SQLite 13
4.4 Dashboard (2/3): occupancy [P] vs camera [M], recorded energy [M], baseline [M] vs controlled [S] cumulative energy 14
4.5 Dashboard (3/3): registered offline experiments from SQLite, live comfort proxies, response time and alerts 15
4.6 Dueling DQN training return over 400 episodes 19

5.1 Occupancy model comparison: F1 by split and inference latency 21
5.2 Test F1 with 95% date-cluster bootstrap intervals and the validation F1 used for selection 22
5.3 Recorded-energy counterfactual savings by occupancy source (Room 1 test dates) 23
5.4 Energy vs comfort trade-off in the closed-loop simulation 24
5.5 One simulated test day: room temperature under each controller 25
5.6 Manual override applied, sent over MQTT and acknowledged in 87 ms 26

List of Tables

2.1 Summary of reviewed literature and relevance to the present study 5

3.1 Hardware and software requirements 7

4.1 ROBOD dataset as used in this project 8
4.2 Reference stages and their implementation 9
4.3 MQTT topics, direction and delivery 10
4.4 SQLite tables 12
4.5 Occupancy and energy models 16
4.6 What was taken from each paper and what was adapted 17
4.7 Experiments and their protocols 20

5.1 Occupancy results, Room 1 (test F1 with 95% date-cluster bootstrap interval) . . 21
5.2 Energy forecasting, next 5 minutes, test dates 22
5.3 Recorded-energy counterfactual, Room 1, 6 test dates, rules (ECO 0.75, DIM 0.5) 23
5.4 Closed-loop simulation, 6 test days (AC only) 24
5.5 Automated tests by area 27

6.1 Team members and subsystems 30

List of Abbreviations

Abbreviation	Expansion
ACL	Access Control List (MQTT broker topic permissions)
AC	Air-Conditioning
ACK	Acknowledgement (actuator feedback message)
BMS	Building Management System
CI	Confidence Interval
CO_2	Carbon dioxide (indoor air-quality indicator)
CV	Cross-Validation
DQN	Deep Q-Network
ECO	Energy-saving AC mode (raised set-point)
F1	Harmonic mean of precision and recall
FCU	Fan Coil Unit
HVAC	Heating, Ventilation and Air-Conditioning
IAQ	Indoor Air Quality
IoT	Internet of Things
K·h	Kelvin-hours (comfort violation magnitude)
LAN	Local Area Network
LSTM	Long Short-Term Memory network
LWT	Last Will and Testament (MQTT)
MAE	Mean Absolute Error
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service (MQTT delivery guarantee)
RC	Resistance–Capacitance (thermal model)
RF	Random Forest
RMSE	Root Mean Squared Error
ROBOD	Room-level Occupancy and Building Operation Dataset

Abbreviation	Expansion
RTT	Round-Trip Time
SSM	State-Space Model
TLS	Transport Layer Security
[M] / [P] / [S]	Measured / Predicted / Simulated value (labels used throughout)

Chapter 1

Introduction

1.1 Background

Lighting and air-conditioning are among the largest energy loads in teaching buildings, and they often run while rooms are empty: between lectures, after the last class, during cancelled sessions. In warm, humid climates the chilled water that cools a lecture room is the dominant load. In the six Room 1 test days of the ROBOD dataset, the room recorded 923 kWh of chilled-water thermal energy and 85.6 kWh of lighting electrical energy, while about 80% of its 5-minute intervals were empty according to the camera ground truth.

The Internet of Things (IoT) makes it possible to sense a room, reason about its occupancy and act on the result automatically. A practical system needs more than a model, though. It needs reliable messaging, integration, storage, visualisation, a safe decision layer, and confirmation that each command was actually executed. This project builds such a system on the reference architecture prescribed by the faculty, using real lecture-room data.

1.2 Problem Statement

The faculty problem statement for Team 1 asks for a system that:

- **monitors** occupancy, temperature, ambient light and electrical power;
- **estimates** occupied and unoccupied periods and energy demand with analytical or machine-learning methods;
- **controls** lighting and AC from those analytics, reducing energy in expected unoccupied periods and/or preparing the room before expected occupancy.

The architecture is prescribed: an IoT node (real data or simulator), MQTT, Node-RED for integration and automation, SQLite or MySQL, and the Node-RED Dashboard.

1.3 Objectives

1. **O1:** implement all eight reference stages, split across a server laptop and an IoT-node laptop.

2. **O2:** compare traditional, ensemble and deep-learning models fairly, on predictive, computational and closed-loop performance.
3. **O3:** integrate two 2024–2026 research algorithms, and state honestly what was adapted.
4. **O4:** control both loads safely, with complete command and acknowledgement feedback.
5. **O5:** report energy and comfort honestly, separating measured, predicted and simulated quantities.
6. **O6:** make the system testable, reproducible, reasonably secure on a LAN, and demonstrable live.

1.4 Scope

In scope:

- two ROBOD lecture rooms replayed as the IoT node;
- occupancy now (presence), next-5-minute energy, and arrival within 30 minutes (pre-cooling);
- lighting (ON/DIM/OFF) and AC (OFF/ECO/COMFORT/BOOST/PRECOOL) control through simulated actuators;
- energy and comfort evaluation in a recorded-energy counterfactual and in a grey-box room simulator.

Out of scope: real relays or BMS integration, metered real-world savings, camera-based people counting, multi-building deployment, TLS certificate management and cloud hosting.

1.5 Significance of the Study

The project puts a complete IoT control loop to work on real building data, and holds itself to one rule on integrity: every number in this report is traceable to a result file produced by the project's own scripts. Every saving is labelled as simulated. Test data was never used for training, model selection or threshold tuning. The same code that runs live is executed by the automated tests and the experiments, so the reported behaviour is the deployed behaviour.

1.6 Methodology Overview

1. The ROBOD data were verified (SHA-256, schema) and split by whole date into training, validation and test dates.

2. Leak-free features were built.
3. Seven occupancy models (including a baseline and the two research models) were trained and compared, together with an energy forecaster and an arrival profile.
4. A Dueling DQN was trained in a grey-box room simulator.
5. The decision engine was implemented once, in Node-RED, and evaluated in closed loop.
6. The full system was run live over MQTT and checked by 97 automated tests and an end-to-end test.

1.7 Organisation of the Report

- Chapter 2 reviews the dataset and the two research papers.
- Chapter 3 sets out the requirements, hardware and software, and the constraints.
- Chapter 4 describes the system design and methods, from MQTT to the Dueling DQN.
- Chapter 5 presents the results: model evaluation, energy and comfort, performance, testing and deployment.
- Chapter 6 concludes with limitations, future work and team contributions.
- The appendix gives the reproduction instructions.

Chapter 2

Literature Review

2.1 ROBOD Dataset

ROBOD (Tekler, Lei, Peng, Miller and Chong, 2022) provides room-level occupancy and building operation data for lecture rooms, offices and a library at the National University of Singapore. It includes Wi-Fi connected devices, CO_2, sound, humidity, temperature and illuminance; plug, lighting, fan and chilled-water energy; and camera-derived occupancy labels. The camera labels make supervised occupancy estimation possible and give an oracle upper bound for control experiments.

2.2 Paper A: OCC-Mamba

I. Qaisar, K. Sun, D. Zhong, X. Yang, X. Miao and Q. Zhao, *OCC-Mamba: A Mamba-based deep learning approach for indoor occupancy prediction*, Building and Environment, article 114085 (2025).

We verified the bibliographic record. We could **not** access the abstract, full text or code: the publisher blocks automated access and no repository is listed. Our implementation therefore follows the **Mamba mechanism** (Gu and Dao, 2023). In Mamba, the state-space step size Δ and the matrices B and C depend on the input, so the recurrence can keep or forget information at each time step, and it runs in linear time. We do not claim OCC-Mamba's configuration, inputs or results.

2.3 Paper B: LSTM and Dueling DQN

F. Hou, J. C. P. Cheng, J. Ma, H. H. L. Kwok, C. Huang and Z. Wu, *Occupancy-driven HVAC control optimization via LSTM and deep reinforcement learning for enhanced indoor air quality, thermal comfort and energy efficiency*, Building and Environment 284 (2025) 113501.

Method (from the published abstract). An LSTM estimates occupancy from indoor CO_2 fluctuations, using labels derived by change-point analysis. A Dueling DQN (Wang et al., 2016)

then reduces HVAC operation in unoccupied periods while maintaining thermal comfort and indoor air quality.

Reported results (theirs, not ours).

- about 0.7 precision and 0.6 recall for occupancy estimation;
- 9.5–14.6% fewer temperature violations;
- 10.9–30.4% less HVAC operation time.

The exact state, action and reward definitions were not accessible to us.

2.4 Gaps and Need for the Study

The two papers are complementary. A is a modern sequence model for the prediction stage. B closes the loop from prediction to HVAC action. Neither describes a complete, reproducible IoT deployment on the prescribed architecture with delivery-confirmed actuation, and neither reports savings next to the comfort cost of wrong decisions in a way that can be checked from public data. This project integrates both methods into such a system, compares them with traditional and ensemble baselines on held-out dates, and reports every saving with its comfort cost.

Table 2.1: Summary of reviewed literature and relevance to the present study

Work	Contribution	Use in this project
Tekler et al. (2022), ROBOD	lecture-room sensor, energy and camera-occupancy dataset	the IoT node's data and the ground truth
Qaisar et al. (2025), OCC-Mamba	Mamba-based occupancy prediction	model family of our deep-learning model A
Gu and Dao (2023), Mamba	selective state-space sequence model	the mechanism we implemented
Hou et al. (2025)	LSTM occupancy + Dueling DQN HVAC control	our LSTM and Dueling DQN controller (B)
Wang et al. (2016)	dueling network architecture	the Q-network head

Chapter 3

Problem Definition and Requirements

3.1 Detailed Problem Description

The system must decide, every five minutes and for each room, whether the room is occupied, and set lighting and AC accordingly. It must do this on the prescribed architecture, over a network between two laptops. Commands must be confirmed, and failures (a lost broker, a missing model, a stale sensor) must be handled safely. Energy savings must be weighed against comfort, because a controller that saves energy by switching off occupied rooms does not solve the problem.

3.2 Project Requirements

3.2.1 Functional Requirements

- Replay real sensor data as an IoT node over MQTT; validate, store and visualise every reading.
- Predict occupancy with several model families; forecast energy; estimate arrivals for pre-cooling.
- Decide lighting and AC commands; send them over MQTT; receive and record acknowledgements.
- Provide manual override, alerts for invalid or stale data, and device-health monitoring.

3.2.2 Analytical Requirements

- Use whole-date splits, so that test dates are never used for training, selection or tuning.
- Compare against baselines, and report predictive, computational and closed-loop metrics.
- Separate measured, predicted and simulated values everywhere.

3.3 Hardware and Software Requirements

Table 3.1: Hardware and software requirements

Item	Laptop 1 (server)	Laptop 2 (node)
OS	Windows 10/11 (Linux/macOS scripts also provided)	Windows 10/11
CPU / RAM / disk	any x86-64, 8 GB RAM, about 3 GB disk	4 GB RAM, about 300 MB
Python	3.11–3.13 (developed and tested on 3.13)	same
Node.js	22 LTS ($\geq$ 20.17); tested on 22.23.4	–
Mosquitto	2.x (tested 2.0.18)	–
Python packages (pinned)	numpy 2.2.6, pandas 2.2.3, paho-mqtt 2.1.0, scikit-learn 1.6.1, scipy 1.15.3, joblib 1.4.2, torch 2.14.1, psutil 7.1.3, matplotlib 3.10.7, pytest 8.4.2	numpy, pandas, paho-mqtt only
Node packages (pinned)	node-red 4.1.17, @flowfuse/node-red-dashboard 1.34.0, node-red-node-sqlite 2.0.1	–
Network	same private LAN or hotspot; TCP 1883 and 1880 open on the Private profile, local subnet only	–

3.4 Constraints and Assumptions

- ROBOD contains no light or AC states, so all actuator states are simulated.
- ROBOD cannot identify the room's thermal plant (linear models explained $R^2 \leq 0.044$ of temperature change), so closed-loop experiments use a grey-box simulator with declared, assumed parameters.
- The dataset licence could not be verified, so the CSV files are downloaded from the official repository and checked against their SHA-256 values, never redistributed.
- The full text of OCC-Mamba was not accessible; paper B's exact state, action and reward definitions were not accessible either.
- The system is a LAN prototype: authenticated MQTT with an ACL, but no TLS.

Chapter 4

System Design and Methods

4.1 Dataset Description

Table 4.1: ROBOD dataset as used in this project

Property	Value
Rooms	Room 1 (primary: training, validation, test); Room 2 (cross-room evaluation only)
Size	8,352 rows × 28 columns per room; 5-minute resolution
Period	29 weekdays: 7 Sep–1 Oct and 9–23 Dec 2021; timestamps kept in +08:00
Integrity	SHA-256 Room 1 `34a3d7a1…3f3d`, Room 2 `ac9a34cd…941f`; checked by `download_data.py` with schema (18 required columns) and row count
Licence	unverified; CSVs not redistributed
Missing data	Room 1: 14 rows without indoor sensors, 10 without chilled-water/fan energy; Room 2: 30 rows

Units. Lighting, FCU fan and plug energy are **electrical** kWh per 5 minutes. Chilled-water energy is **thermal** kWh per 5 minutes and is never added to electrical energy.

Splits (seed 42, by whole date):

- 19 training dates;
- 4 validation dates (2021-09-08, 09-27, 12-09, 12-13), used for model selection, early stopping and threshold tuning;
- 6 test dates (2021-09-20, 09-21, 09-24, 09-30, 12-15, 12-22), used for reporting only.

That gives 5,414 / 1,119 / 1,717 usable rows. Two test dates are almost empty (0 and 1 occupied intervals).

4.2 System Architecture

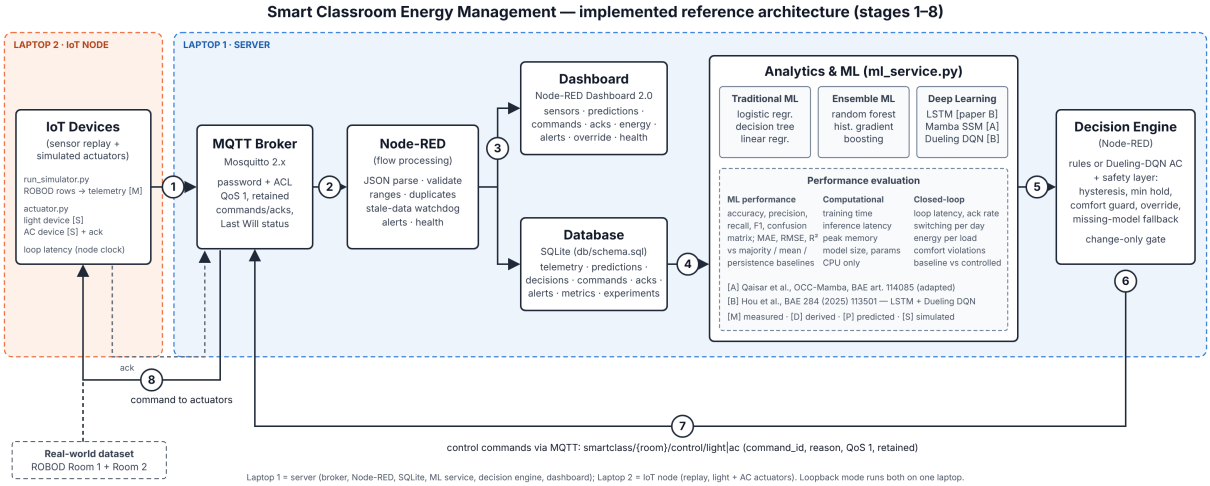


Figure 4.1: Implemented architecture: eight reference stages across two laptops

Table 4.2: Reference stages and their implementation

Stage	Reference block	Implementation	Laptop
1	IoT devices / dataset	`run_simulator.py` (ROBOD rows); `actuator.py` (light + AC per room)	2
2	MQTT broker	Mosquitto: no anonymous access, password file, per-role ACL, QoS 1, retained commands/acks, Last Will	1
3	Node-RED processing	validation, duplicate drop, stale-data watchdog, alerts, device health	1
4	Database + dashboard	SQLite (8 tables); Node-RED Dashboard 2.0	1
5	Analytics and ML	`ml_service.py`: 6 occupancy models, arrival profile, energy forecaster, DQN recommendation	1
6	Decision engine	Node-RED "Decision rules": rules or Dueling DQN behind a safety layer; change-only gate	1
7	Control via MQTT	`smartclass/{room}/control/{light,ac}` with command id	1→2
8	Actuators + feedback	simulated state change; acknowledgement with loop latency	2→1

The control logic exists once, in the Node-RED Decision rules node. The experiments and tests run that exact code. All thresholds live in one CONFIG node, and the database schema exists once,

in db/schema.sql.

4.3 MQTT Architecture

Table 4.3: MQTT topics, direction and delivery

Topic	Direction	QoS / retain	Main fields
smartclass/{room}/telemetry	node → Node-RED	1 / no	sensors, energy [M], camera labels [M], sent_at_ms
smartclass/{room}/validated	Node-RED → ML	1 / no	the validated reading
smartclass/{room}/analytics	ML → Node-RED	1 / no	per-model probabilities [P], arrival, DQN action
smartclass/{room}/control/{light,ac}	Node-RED → actuator	1 / yes	cmd, command_id, reason
smartclass/{room}/actuator/{load}	actuator → Node-RED	1 / yes	state, accepted, command_id, loop latency
…/status	every client	1 / yes	online / offline (Last Will)

Security. Anonymous access is disabled and passwords are stored as PBKDF2-SHA512 hashes. Two roles:

- sc_server can read and write smartclass/#;
- sc_node can only write telemetry, status and acknowledgements, and read control topics.

A node account therefore cannot command actuators; this is tested, and the broker logs Denied PUBLISH. Clients reconnect with 1–30 s exponential back-off.

4.4 Node-RED Flow

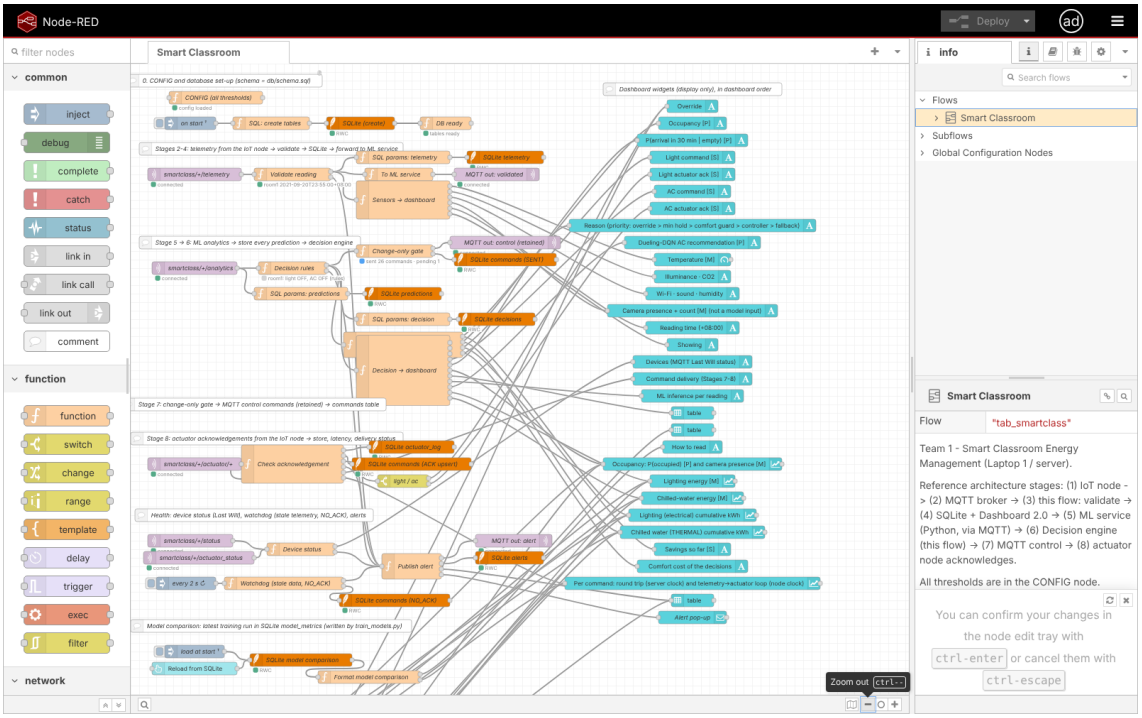


Figure 4.2: Node-RED flow (editor, after login)

The flow (118 nodes, 22 function nodes) is laid out by stage:

1. CONFIG (all thresholds) and table creation from the schema.
2. Telemetry → **Validate reading** (JSON, allowed room, timestamp, ranges, required fields, duplicate drop) → SQLite → `validated`.
3. Analytics → predictions stored → **Decision rules** → decisions stored.
4. **Change-only gate**: a command id is assigned, the command is published retained, and a SENT row is stored.
5. Acknowledgements → `actuator_log` and a command upsert (ACKED, REJECTED or ACKED_LATE), with round-trip and loop latency.
6. Device status (Last Will) and a watchdog every 2 s (stale telemetry after 15 s, NO_ACK after 5 s).
7. Model-performance and offline-experiment tables loaded from SQLite.

Editor security. When an editor password is set, the editor and admin API require a login; otherwise they only answer requests from the server laptop itself.

4.5 Database Design

Table 4.4: SQLite tables

Table	Key columns	Purpose
`telemetry`	UNIQUE(room_id, ts)	every valid reading [M]
`predictions`	room, ts, model, probability, latency	every model output [P]
`decisions`	room, ts, light, ac, reason, controller	every decision [S]
`commands`	command id; status SENT, ACKED, REJECTED, NO_ACK or ACKED_LATE	delivery tracking
`actuator_log`	room, load, state, previous, accepted	every acknowledgement
`alerts`	ts, room, type, message	invalid, stale, offline, no-ack, late-ack
`model_metrics`	run_id, task, model, split, metric, value	offline evaluation (imported)
`experiment_results`	run_id, experiment, scenario, metric, value, unit	registered experiments (imported)

`init_db.py` creates the schema and imports the registered training and experiment results once per run id, without modifying them. Python writers wait up to 15 s for a database lock and raise a visible error after that, rather than dropping a row.

4.6 Dashboard Design

The dashboard implements all twenty required panels. Every value carries a measured [M], predicted [P] or simulated [S] tag, every title states its unit, and live panels state their time window and data source. When data is missing, a panel shows "n/a" or an instruction instead of a misleading zero. Evaluation tables are read from SQLite, not hard-coded. The AC panel states which layer set each command, for example "BOOST — set by rules" and "DQN: OFF — advisory only".

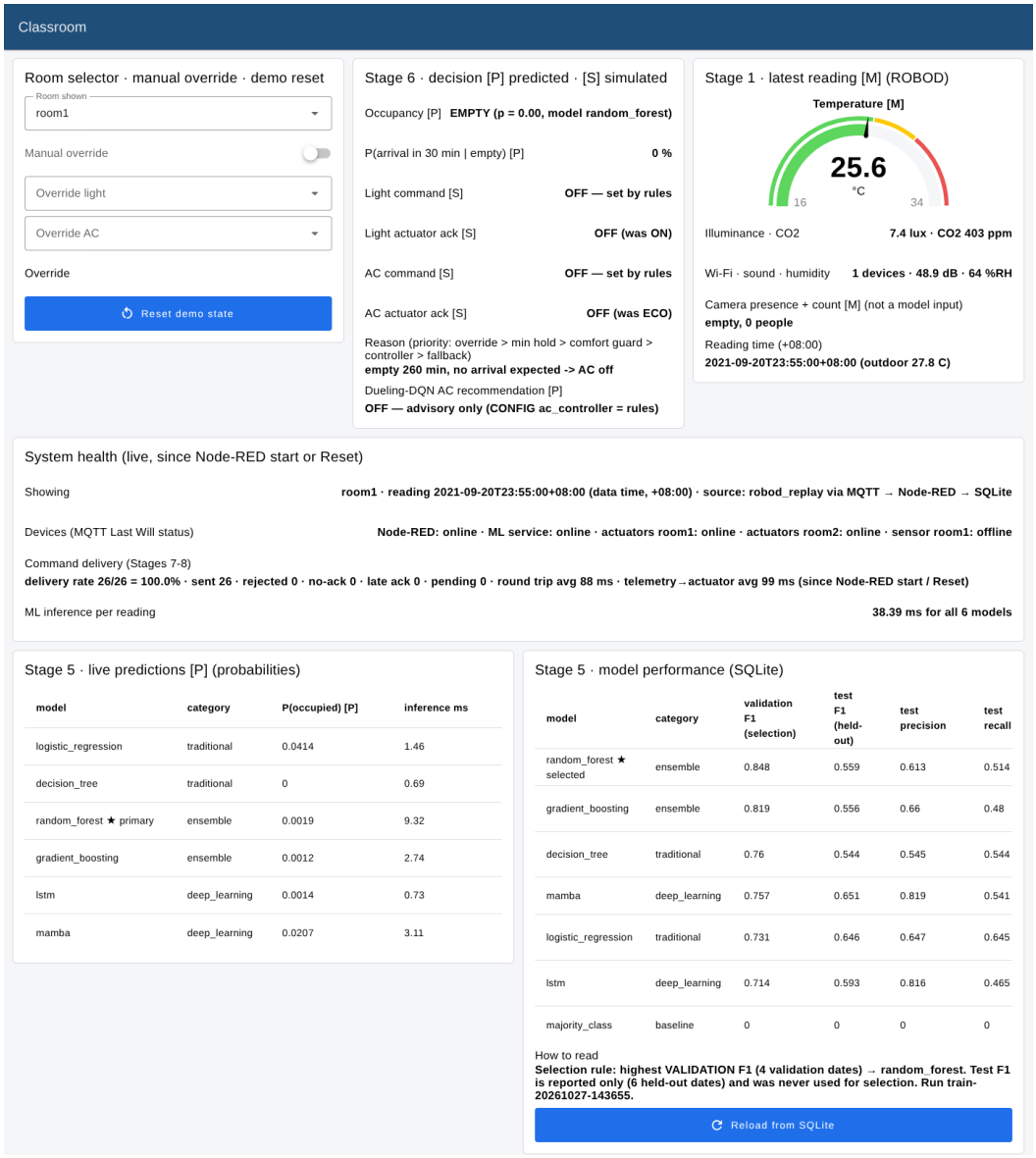


Figure 4.3: Dashboard (1/3): control, decision with the deciding layer, sensors, health, live predictions and model performance from SQLite

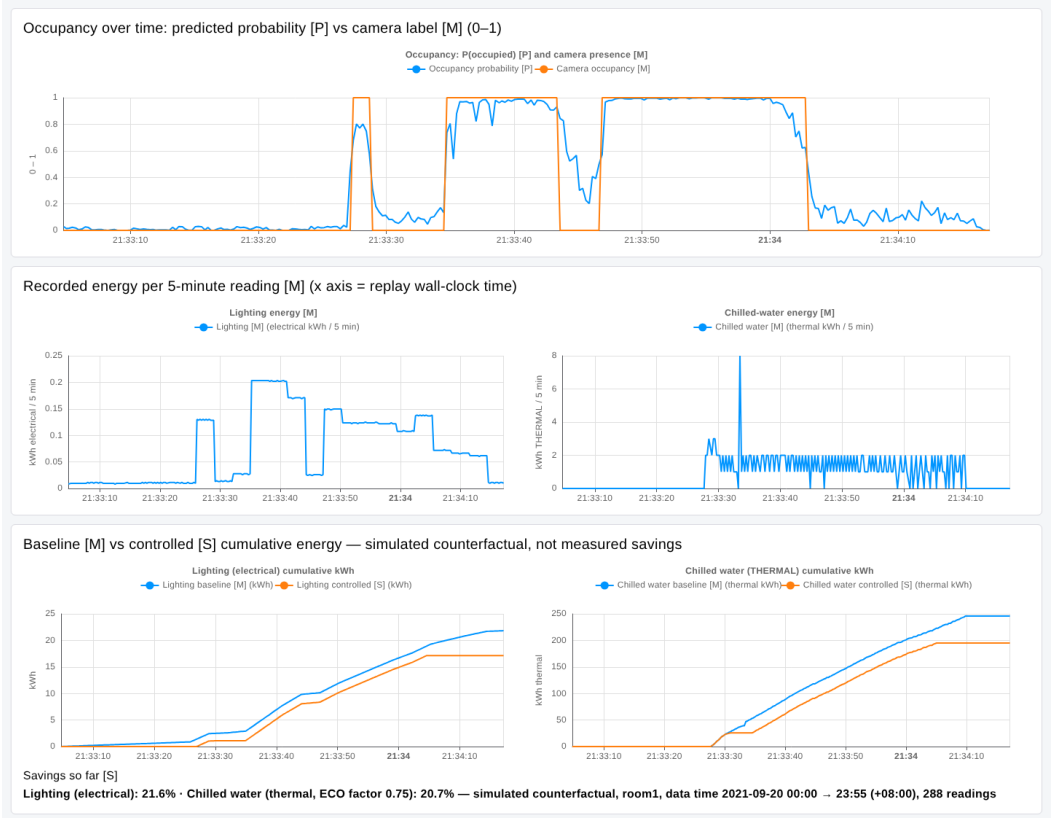


Figure 4.4: Dashboard (2/3): occupancy [P] vs camera [M], recorded energy [M], baseline [M] vs controlled [S] cumulative energy



Figure 4.5: Dashboard (3/3): registered offline experiments from SQLite, live comfort proxies, response time and alerts

4.7 Feature Engineering

- **Sensors** (time t): Wi-Fi devices, CO_2, sound, humidity, temperature, illuminance.
- **Time:** sine and cosine of the time of day, and weekday.
- **Tabular extras:** Wi-Fi 30-minute mean, CO_2 change over 15 and 55 minutes, 30-minute maximum sound, 15-minute temperature change.
- **Sequences:** the last 12 readings (1 hour) of the 9 sensor and time features.

Leakage controls (all tested): features use only time $\leq t$; splits are by whole date; scalers are fitted on training dates only; camera labels are never inputs; live features equal offline features.

4.8 Machine-Learning Models

Table 4.5: Occupancy and energy models

Model	Category	Settings
Majority class	baseline	always predicts "empty"
Logistic regression	traditional	standardised, class-balanced
Decision tree	traditional	depth 5, class-balanced
Random forest	ensemble	200 trees, minimum leaf 5, class-balanced, seed 42
Gradient boosting	ensemble	histogram gradient boosting, 200 iterations, learning rate 0.05
LSTM (paper B)	deep learning	1 layer, 32 units, last hidden state $\rightarrow$ logit; 5,537 parameters
Mamba (paper A family)	deep learning	embedding 32, two pre-norm residual Mamba blocks; 17,217 parameters
Energy forecasters	regression	linear regression, random forest, gradient boosting vs mean and persistence baselines

Deep-learning training. Adam (learning rate 10^{-3}), class-weighted binary cross-entropy and early stopping on validation F1. Each Mamba block has: expand 2, a causal depth-wise convolution (kernel 4), SiLU, input-dependent $\Delta/B/C$ (state size 8), zero-order-hold discretisation, a selective scan, a D skip connection and SiLU gating. It is written in plain PyTorch, with no CUDA kernel.

4.9 Research-Paper Adaptations

Table 4.6: What was taken from each paper and what was adapted

Aspect	Paper	Ours	Why
A: architecture	OCC-Mamba (configuration not accessible)	generic Mamba blocks	full text unavailable
B: occupancy labels	CO_2 change-point labels	ROBOD camera labels	ground truth exists
B: LSTM inputs	CO_2	6 sensors + time	richer signals available
B: DRL state/action/reward	not accessible	our own design	declared openly
B: plant	their simulation platform	grey-box RC thermal + CO_2 model	ROBOD cannot identify the plant
B: deployment	–	DQN behind a safety layer; rules default	comfort results

Neither paper is reproduced exactly; both are research-inspired adaptations.

4.10 Decision Engine

Priority order (highest first), recorded for every decision:

1. manual override;
2. minimum-hold time;
3. comfort safety guard;
4. selected AC controller (rules by default, or the DQN);
5. missing-model fallback.

Rules:

- **Hysteresis:** a room becomes occupied at $p \geq 0.25$ and empty only at $p < 0.15$. These values were tuned on the validation dates; the earlier 0.6/0.4 switched lights off in 22.5% of occupied validation intervals, while 0.25/0.15 does so in 13.7%.
- **Occupied:** light ON (DIM above 300 lux in Room 2 only), AC COMFORT, or BOOST above 25.5 °C.

- **Empty:** a 15-minute grace period, then light OFF and AC ECO; AC OFF after 30 minutes if no arrival is expected.
- **PRECOOL:** when the arrival probability within 30 minutes is at least 0.2 and the room is more than 0.5 °C above comfort.
- **Minimum hold:** 10 minutes for downward switches; switching up for an occupied room is immediate.
- **Start-up:** an empty room is never switched ON.
- **No model output:** hold the last commands, else use the 08:00–20:00 timetable.

The "AC BOOST while the DQN says OFF" case. In the default rules mode the DQN is advisory only. The rules chose BOOST because the room was occupied above 25.5 °C, and the dashboard now says so. The safety layer is never bypassed in favour of the DQN.

4.11 Dueling DQN

- **Network:** $Q(s,a) = V(s) + A(s,a) - \overline{A}(s)$, with two hidden layers of 64 ReLU units; 24 kB on disk; 0.12 ms per decision.
- **State:** LSTM occupancy probability, arrival probability, temperature, CO_2, outdoor temperature, time of day.
- **Actions:** OFF, ECO, COMFORT, BOOST.
- **Reward per 5 minutes:** $-(\text{cooling kWh}/3.5 + \text{fan kWh}) - 0.5 \times \text{K outside 23–26 °C} - 0.3 \times (\text{CO}_2 - 1000)/100$. The comfort and CO_2 terms apply only while the room is occupied.
- **Training:** replay buffer 50k, target network synchronised every 1,000 steps, ε from 1 to 0.05, $\gamma = 0.97$, Huber loss, 400 one-day episodes on training and validation dates (219 s on CPU). The occupancy input is out-of-fold LSTM output, to avoid in-sample optimism.

Simulator. A first-order RC model with an ideal thermostat per mode (ECO 26, COMFORT 24, BOOST 23 °C) and a CO_2 mass balance, driven by ROBOD's outdoor temperature, solar radiation, outdoor CO_2 and occupant counts. The parameters are assumed and declared, and were calibrated so that the free-running room peaks around 30 °C and 24/7 COMFORT uses about 140 kWh thermal per day (ROBOD median 132).

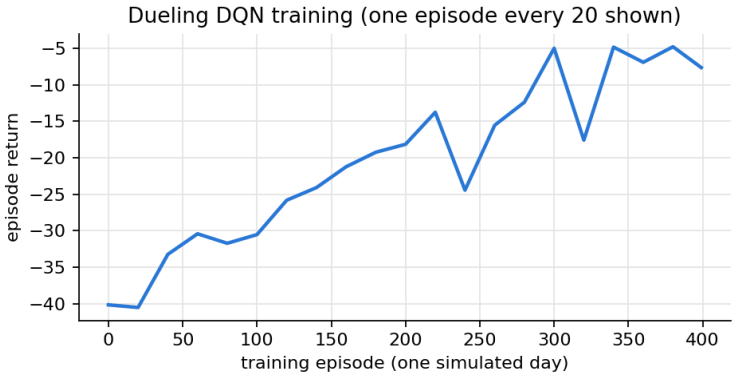


Figure 4.6: Dueling DQN training return over 400 episodes

4.12 Actuator Simulation

`actuator.py` runs on Laptop 2, one process per room, controlling two loads: light (ON/DIM/OFF) and AC (OFF/ECO/COMFORT/BOOST/PRECOOL). For each command it validates the value, applies it to the simulated state, and publishes a retained acknowledgement with the state, previous state, acceptance, command id and loop latency (measured on the node's own clock). An unknown command is rejected and raises an alert. No physical actuator was tested.

4.13 Experimental Methodology

Table 4.7: Experiments and their protocols

Experiment	Protocol
Occupancy models	train on 19 dates; select and tune on 4 validation dates; report on 6 test dates; 5-fold GroupKFold by date; Room 2 cross-room
Test-set uncertainty	saved models re-predict the test dates (F1 reproduced exactly); 2,000× date-cluster bootstrap (seed 42), paired difference vs the primary model
Energy forecasting	next 5 minutes, test dates, vs mean and persistence baselines
Recorded counterfactual	recorded energy × our command (light OFF 0, DIM 0.5; AC OFF 0, ECO 0.75), per occupancy source; sensitivity ECO 0.5–0.9
Closed-loop simulation	6 test days in the grey-box room; always-on, timetable, rules, raw DQN, DQN + safety; occupancy from camera, LSTM, Mamba or RF
DQN reward weight	150 episodes, camera occupancy, raw policy, 6 test dates (shortened protocol)
Live closed loop	full-day replay through MQTT, Node-RED, ML and actuators; metrics from SQLite

Test dates were never used for training, selection or tuning, and all seeds are 42.

Chapter 5

Results and Discussion

5.1 Model Evaluation

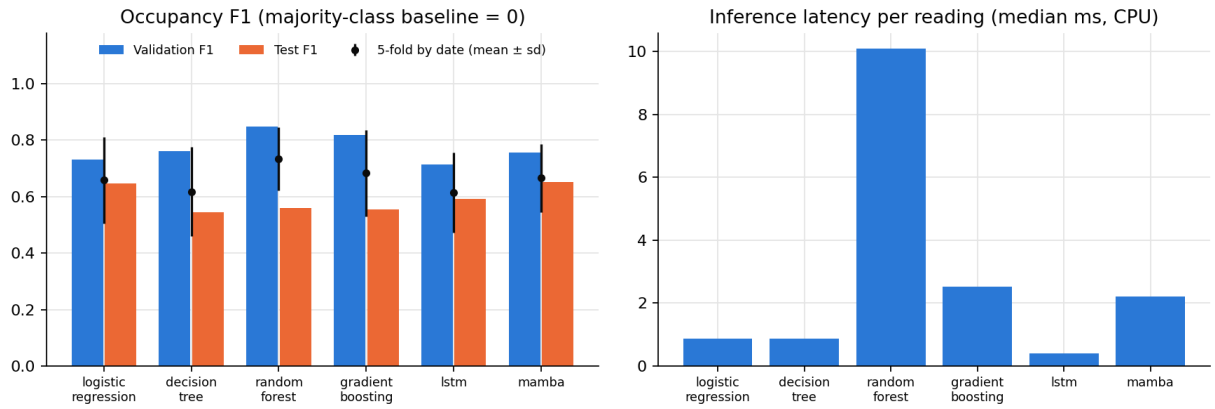


Figure 5.1: Occupancy model comparison: F1 by split and inference latency

Table 5.1: Occupancy results, Room 1 (test F1 with 95% date-cluster bootstrap interval)

Model	Val F1	Test acc.	Test P / R	Test F1 [95% CI]	CV F1	Room 2 F1	Latency (ms)
Majority class	0.000	0.800	0 / 0	0.000	0.00	0.000	0.02
Logistic regression	0.731	0.859	0.65 / 0.65	0.646 [0.34, 0.77]	0.66 ± 0.15	0.679	0.88
Decision tree	0.760	0.818	0.55 / 0.54	0.544 [0.20, 0.77]	0.62 ± 0.16	0.657	0.86
Random forest	**0.848**	0.838	0.61 / 0.51	0.559 [0.19, 0.78]	**0.73 ± 0.11**	0.667	10.1
Gradient boosting	0.819	0.846	0.66 / 0.48	0.556 [0.14, 0.77]	0.68 ± 0.15	0.635	2.5
LSTM (B)	0.714	0.872	0.82 / 0.47	0.593 [0.06, 0.81]	0.61 ± 0.14	0.637	0.40
Mamba (A)	0.757	**0.884**	0.82 / 0.54	**0.651** [0.30, 0.78]	0.67 ± 0.12	0.570	2.2

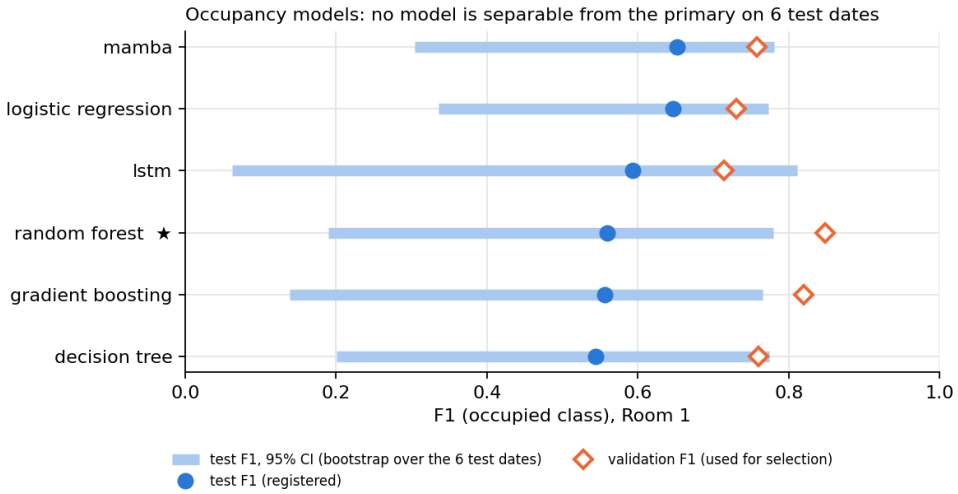


Figure 5.2: Test F1 with 95% date-cluster bootstrap intervals and the validation F1 used for selection

Why the random forest is the primary model.

- It was chosen by the rule fixed in advance: the highest validation F1 (0.848). It also had the best 5-fold CV F1 (0.73 ± 0.11).
- Mamba scores higher on test, but re-selecting by test F1 would be test-set selection, which would make the test score optimistic.
- The paired bootstrap shows the gap is not significant: Mamba $-$ RF $= +0.093$ F1, 95% CI $[-0.18, +0.38]$. Every model's interval against the random forest includes zero.
- Two test days have 0 and 1 occupied intervals, which widens the intervals.

All models are far above the majority baseline. The deep models are the most precise (0.82) but miss about half of the occupied intervals. Every model runs in at most 15 ms per reading (p95) on one CPU thread.

Table 5.2: Energy forecasting, next 5 minutes, test dates

Model	Lighting MAE (kWh)	Lighting R^2	Chilled-water MAE (kWh th)	Chilled-water R^2
Mean baseline	0.0539	-0.00	0.669	-0.02
Persistence	**0.0022**	0.965	0.450	-0.09
Linear regression	0.0028	**0.966**	0.466	0.32
Random forest	0.0144	0.836	0.244	0.66
Gradient boosting	0.0040	**0.966**	**0.243**	**0.70**

For lighting, nothing beats persistence: the lights rarely change between 5-minute steps. For chilled water, the ensembles halve the error of both baselines. The pre-cooling arrival profile caught 7 of 47 arrivals with 158 false alarms (F1 0.066): it works as designed, but it is weak.

5.2 Energy Evaluation

All numbers in this section are **simulated**; none is a measured saving in a real building.

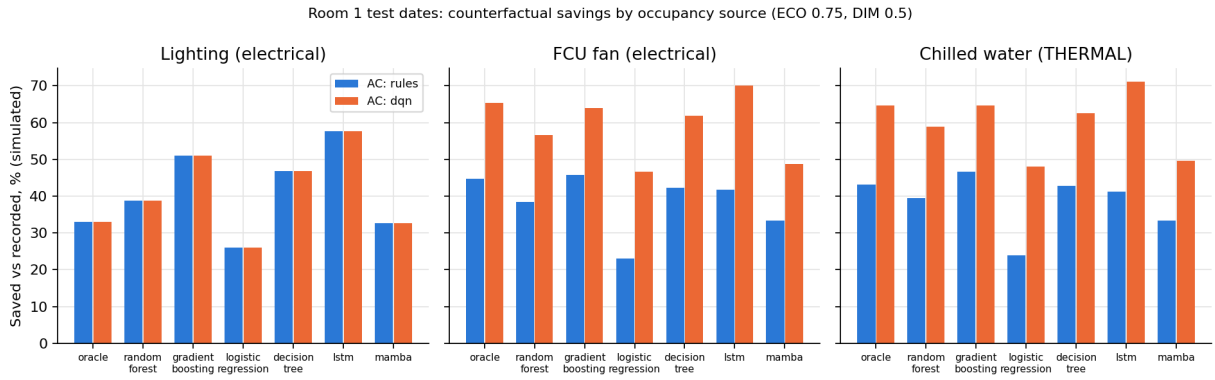


Figure 5.3: Recorded-energy counterfactual savings by occupancy source (Room 1 test dates)

Table 5.3: Recorded-energy counterfactual, Room 1, 6 test dates, rules (ECO 0.75, DIM 0.5)

Occupancy input	**Lighting**	**Chilled water (th)**	**FCU fan**	**Lights OFF while occupied**
Camera label (upper bound)	33.0%	43.1%	44.7%	0 / 344
Random forest (deployed)	38.7%	39.5%	38.4%	96 / 344 (27.9%)
Logistic regression	26.1%	23.9%	23.0%	36 / 344 (10.5%)
Mamba	32.6%	33.4%	33.3%	64 / 344 (18.6%)
LSTM	57.7%	41.2%	41.8%	133 / 344 (38.7%)

Savings above the camera bound come from switching off occupied rooms; they are a comfort cost, not a real saving. Chilled-water savings change by less than 1.5 points across ECO factors 0.5–0.9, because most of the saving comes from switching the AC off.

Table 5.4: Closed-loop simulation, 6 test days (AC only)

Controller	Cooling kWh_{th}	vs always-on	Fan kWh_e	Outside 23–26 °C	K·h	Switches/day
Always COMFORT	850.6	–	37.5	0.0%	0.00	0.0
Timetable 07–20	509.5	−40.1%	20.3	0.0%	0.00	2.0
Rules + camera (bound)	366.7	−56.9%	12.3	0.3%	0.01	14.7
Rules + LSTM	361.7	−57.5%	12.3	0.6%	0.07	17.0
Rules + Mamba	422.9	−50.3%	15.1	1.2%	0.08	14.0
Rules + RF (default)	**405.5**	**−52.3%**	14.2	**0.6%**	0.07	16.5
DQN raw + LSTM	128.5	−84.9%	2.3	24.1%	3.13	25.3
DQN + safety + LSTM	150.9	−82.3%	2.8	7.0%	0.59	17.7
DQN + safety + RF	152.8	−82.0%	2.7	12.2%	1.20	17.3

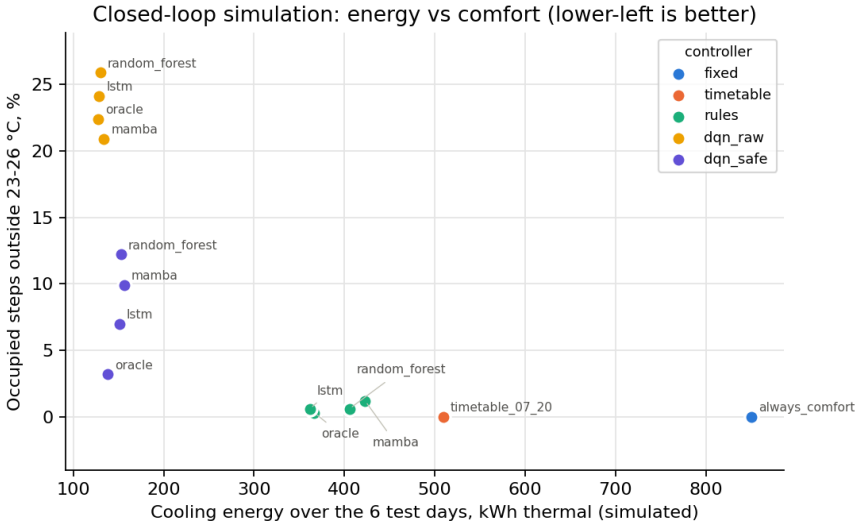


Figure 5.4: Energy vs comfort trade-off in the closed-loop simulation

5.3 Comfort Analysis

- **Rules:** 0.6% of occupied intervals fall outside 23–26 °C with the deployed model, close to the 0.3% camera bound. CO_2 never exceeded 1,000 ppm in any scenario, so the air-quality term was never binding.
- **Lighting** is the real comfort weakness of predicted occupancy: lights were wrongly OFF in 27.9% of occupied intervals on the test dates with the deployed model (13.7% on validation, 8.1% on Room 2). A deployment would need a motion sensor or a minimum-lighting fallback.

- **DQN:** the raw policy leaves occupied rooms warm (24.1% violations); the safety layer cuts this to 7.0%. Raising the comfort weight from 0.5 to 2.0 (shortened protocol) cut violations from 26.5% to 1.7% for 5% more cooling (115.7 $\rightarrow$ 121.6 kWh thermal). The problem is the reward weighting.
- **Live:** during the 2021-09-20 replay, 0 of 105 occupied readings had the lights OFF or the AC in ECO/OFF.

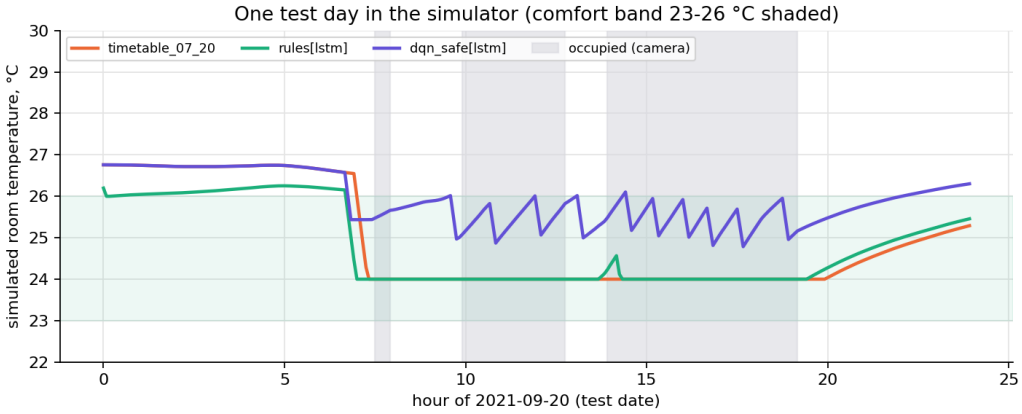


Figure 5.5: One simulated test day: room temperature under each controller

5.4 Performance Evaluation

Live run (one machine, through its network interface):

- 290 readings (288 replay + 2 demonstration messages), 290 decisions, 6 $\times$ 290 predictions;
- 29 commands, 100% acknowledged;
- round trip: median 88 ms, p95 92 ms, max 93 ms;
- telemetry-to-actuator loop: median 94 ms, p95 159 ms;
- per-model inference 1.0–12.7 ms, and 38–54 ms for all six models per reading.

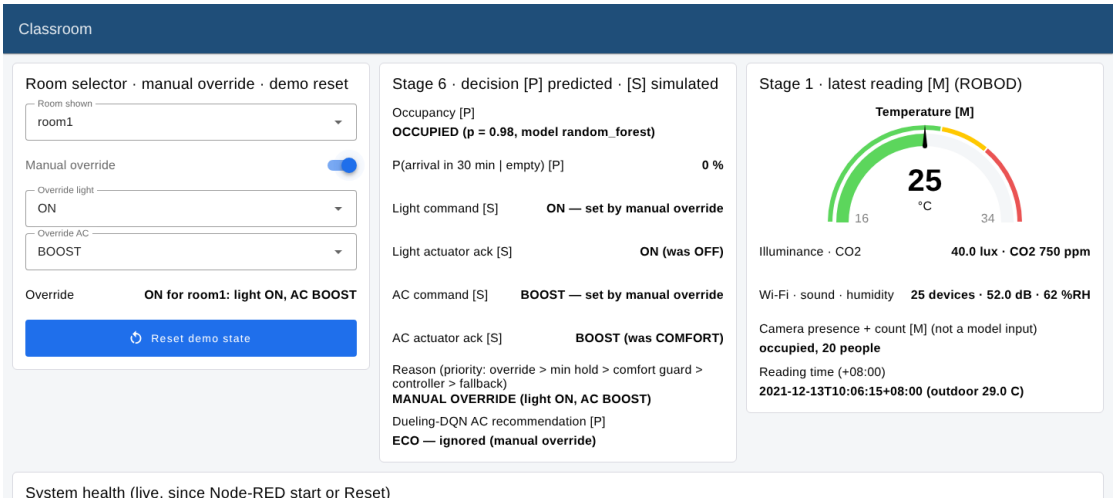


Figure 5.6: Manual override applied, sent over MQTT and acknowledged in 87 ms

5.5 Testing

The command `python -m pytest -q` gives **97 passed, 1 skipped** (the skip is the opt-in end-to-end test). The end-to-end test, run separately, also passes: real broker, Node-RED, ML service, actuator and replay, with 100% of commands acknowledged. A clean-room run from the delivered ZIP, with a fresh official dataset download, gave the same results.

Table 5.5: Automated tests by area

Test file	Tests	Covers
test_data_features	9	schema, units, timestamps, no silent filling, date splits, no future information, live = offline features
test_models_eval	7	every category trains and predicts, save/load, seeded training, dueling head, report consistency, fallback
test_messages_actuator	4	strict JSON telemetry, actuator accepts/rejects, separate devices
test_flow_functions	19	validation, duplicates, stale data, gate ids, ACK, savings and comfort counters, SQLite-driven panels
test_rules	14	hysteresis, grace, AC off, BOOST, pre-cool, hold, start-up, override, DQN + safety, displayed decision source
test_config_flows	13	flow wiring, schema = flow, no secrets in flows, pinned packages, broker ACL
test_failures	19	broker down/restart, wrong credentials, ACL, ML crash, missing model, DB locked, late/unknown ACK, dataset tampering
test_scripts	12	setup/start/stop scripts, shell and PowerShell parsing, firewall scope
test_e2e_loopback (opt-in)	1	full system with 60 readings

Testing found and fixed four real defects:

- duplicate command ids within one millisecond;
- late acknowledgements after the timeout being ignored;
- a stop script returning before Node-RED had exited;
- an editor session file tracked by version control.

5.6 Two-Laptop Deployment

Laptop 1 runs `setup_server.ps1`, then `open_firewall.ps1` (once, as Administrator), then `start_server.ps1`. Laptop 2 runs `setup_node.ps1 -ServerIp <IP>`, then `start_node.ps1`. Shutdown uses `stop_node.ps1` and `stop_server.ps1`.

The scripts:

- detect Python, Node.js and Mosquitto, and install the dependencies;
- generate random credentials and validate the environment and the dataset hash;
- refuse to start on a busy port, naming the owning process;

- print the LAN IP, test TCP and MQTT login from the node, and record process ids for clean shutdown.

Verification status. The PowerShell scripts pass the real PowerShell parser, and the Linux equivalents ran the complete system. Operation on two separate physical laptops is prepared, with a 15-item checklist, but had not been verified at the time of writing.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The system implements all eight reference stages: authenticated MQTT, Node-RED integration, SQLite, a Node-RED dashboard that reads its evaluation results from the database, and a complete command and acknowledgement loop. Delivery is measured: 100% of commands were acknowledged, with a round trip of about 0.09 s.

Traditional, ensemble and deep models, including the two research adaptations, were compared on held-out dates without test-set selection. Mamba scored highest on test, but the differences are within the date-to-date uncertainty, and the validation-selected random forest remains the primary model.

Occupancy-driven rules halved simulated cooling energy at under 1% comfort violations. The paper-B Dueling DQN saves more energy but needs a stronger comfort weight, so it stays optional behind a safety layer.

Every number in this report is traceable to a result file, and every saving is a simulation result.

6.2 Limitations

1. **Small data:** 29 dates, of which 6 are test dates (two nearly empty), so model rankings are not statistically separable.
2. **OCC-Mamba** is followed only at the model-family level.
3. **Grey-box plant** with assumed parameters: absolute closed-loop energy is indicative only.
4. **Counterfactual energy** assumes energy scales with our command, so savings are optimistic.
5. **Lighting comfort:** lights are wrongly OFF in 8–28% of occupied intervals, depending on room and split.
6. **DQN:** the comfort weight is too low and switching is frequent.
7. **Security:** no TLS on MQTT, and the dashboard controls have no login on the LAN.

8. **Not verified:** two physical laptops, physical actuators and the dataset licence.

6.3 Future Work

- Add a PIR motion sensor or a minimum-lighting fallback to remove wrong lights-OFF.
- Tune the DQN reward (comfort weight, switching penalty, previous action in the state) on validation dates.
- Use more dates and rooms for stable model selection, and report confidence intervals as standard.
- Add TLS for MQTT and a login for the dashboard controls.
- Obtain the OCC-Mamba full text to align the architecture.
- Run a pilot in a real room with relays and metered energy.

6.4 Team Contributions

The decision engine, Dueling DQN and energy evaluation is the largest machine-learning part of the project, and was shared by two members.

Table 6.1: Team members and subsystems

Member	Roll Number	Subsystem
NIRANJAN N S	AM.SC.U4CSE23338	decision engine, Dueling DQN, energy and comfort evaluation (shared with Sivadev Smijith)
SIVADEV SMIJITH	AM.SC.U4CSE23359	decision engine, Dueling DQN, energy and comfort evaluation (shared with Niranjan N S); Node-RED integration, SQLite, dashboard
YALAMARTHI JYOTHI SRI KRISHNA CHAITANYA	AM.SC.U4CSE23370	IoT node, MQTT broker, security, two-laptop deployment
PENUMALLU RAM DINESH REDDY	AM.SC.U4CSE23341	dataset, features, traditional and ensemble ML, evaluation
MANISH SHISHIRCHANDRA DORA ALURI	AM.SC.U4CSE23333	LSTM and Mamba models, ML service

References

[1] Z. D. Tekler, Y. Lei, Y. Peng, C. Miller and A. Chong, "ROBOD, room-level occupancy and building operation dataset," *Building Simulation*, vol. 15, no. 12, 2022. doi:10.1007/s12273-022-0925-9. Data: https://github.com/ideas-lab-nus/robod

[2] I. Qaisar, K. Sun, D. Zhong, X. Yang, X. Miao and Q. Zhao, "OCC-Mamba: A Mamba-based deep learning approach for indoor occupancy prediction," *Building and Environment*, article 114085, 2025. doi:10.1016/j.buildenv.2025.114085 (record verified; full text not accessed).

[3] F. Hou, J. C. P. Cheng, J. Ma, H. H. L. Kwok, C. Huang and Z. Wu, "Occupancy-driven HVAC control optimization via LSTM and deep reinforcement learning for enhanced indoor air quality, thermal comfort and energy efficiency," *Building and Environment*, vol. 284, 113501, 2025. doi:10.1016/j.buildenv.2025.113501

[4] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[5] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016. arXiv:1511.06581

[6] K. Sun, I. Qaisar, M. A. Khan, T. Xing and Q. Zhao, "Building occupancy number prediction: A Transformer approach," *Building and Environment*, vol. 244, 110807, 2023.

[7] OASIS, *MQTT Version 3.1.1*, OASIS Standard, 2014.

[8] Eclipse Foundation, *Eclipse Mosquitto documentation* (mosquitto.conf, password file and ACL).

[9] OpenJS Foundation, *Node-RED 4 documentation*; FlowFuse, *Node-RED Dashboard 2.0 documentation*.

Appendix A

Reproduction Instructions

```
# Laptop 1 (Linux/macOS; Windows: the .ps1 equivalents)
bash scripts/setup_server.sh          # venv, deps, .env with random passwords,
                                      # dataset + SHA-256, models, npm, broker users, DB
python download_data.py --verify      # PASS x2: sha256 matches, 8352 rows, schema ok
python train_models.py                # ~10 min CPU; models/ and results/ (seed 42)
python run_experiments.py             # counterfactual + closed loop; results/*.csv
python tools/bootstrap_ci.py          # supplementary confidence intervals
python -m pytest -q                   # 97 passed, 1 skipped
SC_E2E=1 python -m pytest -q tests/test_e2e_loopback.py   # 1 passed
bash scripts/start_server.sh          # then on Laptop 2:
bash scripts/start_node.sh --date 2021-09-20
```

Versions and configuration:

- Python 3.13.17, Node.js 22.23.4, Mosquitto 2.0.18, node-red 4.1.17; Python packages pinned in `requirements*.txt`.
- Seeds 42 throughout.
- Splits and features are recorded in `models/training_report.json`; the model manifest is in `models/final/manifest.json`.
- Thresholds are in the Node-RED CONFIG node; simulator parameters are in `smartclass/room_env.py`.